

Automatiserad rapportgenerering med Jinja2 och WeasyPrint

Magdalena Persson

Syftet med mitt projekt var att lära mig hur man kan bygga ett Pythonprogram för automatiserad rapportgenerering, där data läses in, bearbetas och analyseras för att sedan presenteras i en färdig rapport.

Jag ville få en bättre förståelse för hur programmets olika delar kan struktureras och kopplas samman, från inläsning, rensning och validering av data till beräkning av KPI, visualisering och rapportgenerering. Ett viktigt mål var att förstå hur programmet kan byggas så att samma kod kan användas när ny data kommer in.

Projektet skulle även demonstrera hur Jinja2 kan användas för att skapa en dynamisk rapportmall och hur WeasyPrint kan användas för att omvandla HTML till en PDF-rapport. Ett viktigt mål var att förstå hur programmet kan byggas så att samma kod kan användas när ny data kommer in.

Jag valde att arbeta med automatiserad rapportgenerering, med en fördjupning i Jinja2 och WeasyPrint. Området kombinerar databehandling, analys, visualisering och presentation av resultat.

Jag har blivit särskilt intresserad av hur data kan användas för att skapa tydliga beslutsunderlag. Jag tyckte mycket om kursen i Business Intelligence, där fokus låg på att analysera data och sedan presentera resultat på ett sätt som kan användas i en verksamhet. Därför kändes automatiserad rapportgenerering som ett intressant område att fördjupa mig inom.

Området är relevant för Data Science och närliggande roller som dataanalytiker och digital analytiker, där data ofta behöver bearbetas, analyseras och presenteras återkommande. Genom att automatisera delar av rapporteringen kan manuella moment minska och samma analys kan återanvändas när ny data kommer in.

Jinja2 – Ett templating-bibliotek för Python som gör det möjligt att skapa dynamiska HTML-sidor. I mitt projekt används Jinja2 för att skapa rapporten utifrån en HTML-mall och data från Python.

Template och base.html – En template är en återanvändbar mall. `base.html` innehåller rapportens gemensamma struktur och CSS, medan `report.html` bygger vidare på mallen genom så kallad template inheritance.

get_template() och render() – `get_template()` används för att hämta den HTML-mallen och `render()` används för att skicka data från Python till mallen.

Jinja2-variabler och loopar – Jinja2 använder speciell syntax för att lägga in dynamiskt innehåll i HTML. Exempelvis används `{{ total_revenue }}` för att visa värde från Python. Loopar kan användas för att gå igenom exempelvis flera rader i en KPI-tabell.

CSS (Cascading Style Sheets) – CSS används för att styra i HTML-rapportens utseende, exempelvis färger, typsnitt, tabeller, avstånd och layout.

WeasyPrint – WeasyPrint används för att omvandla den färdiga HTML-rapporten till en PDF-fil.

Jag använde datasetet `web_traffic_marketing_data_2026.csv`, som togs fram för projektet med fokus på webbtrafik och digital marknadsföring. Datasetet innehåller information om datum, trafikälla, enhet, sessioner, användare, sidvisningar, avvisningsfrekvens, genomsnittslig sessionstid, konverteringar, inäkter och marknadsföring.

En rad i datasetet motsvarar en kombination av datum, trafikälla och enhet. Datasetet innehöll från början vissa saknade värden och datatyper som behövde justeras, vilket gjorde det lämpligt att arbeta med datarensning och validering.

Projektet utvecklades i Python. Jag använde **pandas** för databehandling, **Pathlib** för sökvägar, **Matplotlib** för visualiseringar, **pytest** för tester samt **Jinja2** och **WeasyPrint** för rapportgenereringen.

Jag valde att dela upp programmet i flera olika moduler med olika ansvarsområden. Data läses först in och rensas och valideras. Därefter beräknas KPI och visualiseringar skapas. Resultaten skickas till en Jinja2-mall som skapar HTML-rapporten, vilken sedan omvandlas till PDF med hjälp av WeasyPrint.

För rapportgenereringen användes Jinja2 för att kombinera analysresultaten med en färdig HTML-mall. En gemensam `base.html` innehåller rapportens övergripande struktur och layout, medan `report.html` innehåller det specifika rapportinnehållet. På så sätt kan samma mall återanvändas när nya resultat genereras. Den färdiga HTML-rapporten konverteras sedan till PDF med WeasyPrint.

Kedjan kan sammanfattas som:

CSV → inläsning → rensning → validering → KPI → visualiseringar → Jinja2 → HTML → WeasyPrint → PDF

En enda körning av `main()` genomför hela kedjan automatiskt.

Jag valde att ta fram KPI inom tre områden: kanalanalys, enhetsanalys och tidsanalys.

För kanalanalysen beräknades bland annat sessioner, konverteringar, intäkter, konverteringsgrad, intäkt per session och ROAS. För enhetsanalysen sammanställdes motsvarande nyckeltal för olika typer av enheter. Tidsanalysen användes bland annat för att följa hur intäkterna utvecklades över tid.

Eftersom datasetet innehöll vissa saknade värden och datatyper som behövde justeras använde jag även pytest för att testa funktionerna. Jag testade bland annat att data rensas och konverteras till rätt datatyper samt att funktionerna hanterar olika typer av indata och gränsfall. Rensning och validering separerades i olika funktioner för att tydliggöra deras olika ansvarsområden.

Resultatet blev en fungerande Pythonlösning som automatiskt bearbetar data och genererar en färdig rapport i både HTML- och PDF-format. Rapporten innehåller övergripande KPI, visualiseringar och en tabell med resultat uppdelade efter enhet.

Den färdiga rapporten visar bland annat totala sessioner, konverteringar, intäkter och konverteringsgrad samt visualiseringar av intäkt i förhållande till marknadsföringskostnad, ROAS per kanal och intäktsutveckling över tid.

En viktig del av resultatet är att samma program kan köras igen med ny data och generera en uppdaterad rapport utan att Pythonkoden behöver ändras.

Två bilder på rapport med olika resultat..

Kodexempel:

```
<h3>Total Sessions</h3>
```

```
<p>{{ total_sessions }}</p>
```

Värdet total_sessions hämtas från Python och sätts automatiskt in i HTML-mallen när rapporten genereras.

En begränsning med den nuvarande lösningen är att CSV-filen måste uppdateras manuellt inför varje körning. Programmet identifierar inte automatiskt en ny datafil och hämtar inte heller data direkt från ett externt system.

En möjlig förbättring skulle kunna vara att automatiskt identifiera den senaste CSV-filen eller att hämta data direkt från ett API eller en databas. En annan utvecklingsmöjlighet är att lägga till schemalagd körning, exempelvis med Windows Task Scheduler, så att rapporten automatiskt kan genereras varje dag eller vecka.

Rapporten är även relativt enkel i sin nuvarande form och skulle kunna utvecklas med fler KPI, analyser och möjlighet att välja exempelvis tidsperiod och kanal.

Automatiserad rapportgenerering kan användas i flera dataroller där data behöver sammanställas och presenteras återkommande. Den data scientist eller dataanalytiker kan exempelvis använda Python för att automatisera delar av arbetet med att bearbeta och analysera data samt skapa återkommande rapporter.

För en digital analytiker kan en liknande lösning användas för exempelvis webbtrafik, konverteringar och marknadsföringsdata. Genom att automatisera återkommande rapportering kan manuella moment minska och mer tid läggas på att analysera resultaten och dra slutsatser av dem.

Jinja2 och WeasyPrint visar även hur programmering kan användas för att gå från analyserad data till en färdig och strukturerad rapport.

Självreflektion

Under projektets gång fick jag en grundläggande förståelse för HTML och hur Jinja2 kan användas tillsammans med Python för att skapa dynamiska rapporter. WeasyPrint var också helt nytt för mig. Jag fick dessutom en bättre förståelse för hur olika tekniker samverkar, där Python bearbetar data, Jinja2 bygger HTML-rapporten och WeasyPrint omvandlar den till PDF.

Det svåraste var att förstå hur Jinja2 kopplas ihop med Python och hur data skickas från Python till HTML-mallen. Nu kan jag följa och förstå lösningen bättre, men behöver fortfarande öva på att skriva mer av koden själv från grunden.

Det tekniska val jag är mest nöjd med är att dela upp projektet i olika moduler med tydliga ansvarsområden. Det gjorde lösningen lättare att följa och bygga vidare på.

Om jag började om hade jag använt en CSV-fil med data som inte behövde rensas för att kunna lägga mer tid på automatisering, exempelvis automatisk datainläsning och schemalagd rapportgenerering. Ett naturligt nästa steg skulle därför vara att hämta data från en databas eller ett API och automatiskt skapa rapporten enligt ett schema. Jag skulle även vilja undersöka möjligheten att skapa en interaktiv version av rapporten.

Jag tycker att jag uppfyllt kraven för G genom att genomföra en tydligt avgränsad fördjupning och bygga en fungerande och strukturerad lösning. Samtidigt tycker jag att jag gått lite djupare inom Jinja2 och WeasyPrint genom att försöka förstå hur de olika delarna fungerar tillsammans med Python, gjort egna tekniska val och funderat kring vad som skulle kunna utvecklas vidare. Jag har fortfarande mycket att lära mig inom området, men projektet har gett mig en bra grund och en vilja att fortsätta fördjupa mig.

Källor

- [Jinja — Jinja Documentation \(3.1.x\)](#) – Användes som teknisk dokumentation för Jinja2, bland annat mallar och template inheritance.
- [Automated reports using python and jinja HTML templates](#) – YouTube. Användes som stöd för arbetet med Jinja2 och automatiserad rapportgenerering.
- [MSYS2](#) – Användes som stöd för installationen av de externa bibliotek som behövdes för att köra WeasyPrint i Windows.
- [WeasyPrint – Advanced HTML and CSS to PDF Generator](#) – Användes som documentation för WeasyPrint och installation av biblioteket.
- OpenAI - [ChatGPT](#) – Användes som stöd under projektets arbetsprocess för idéutveckling, problemlösning, felsökning och diskussion kring kodstruktur.